



Bicol University
College of Science
Information Technology Department
A.Y: 2025-2026



FITMATE: Gym Member Management System

By:

Malate, Jessa N.
Napolis, Ma. Carmilla N.

Presented to:

Montas, Jevah Rea

Presented by:

Colina, Trisha Mae M.
Mayor, Jon Wilfred C.
Ocfemia, Miko N.
Pura, Althea Stephanie P.

1. Project Overview

The FITMATE Gym Member Application and Profile Management System is a digital administrative module designed to streamline gym operations. Developed by Trisha Mae M. Colina, Jon Wilfred C. Mayor, Miko N. Ocfemia, and Althea Stephanie P. Pura, the project builds upon a database design schema originally formulated by Jessa N. Malate and Ma. Carmilla N. Napolis. Presented to Instructor Jevah Rea Montas at Bicol University's College of Science, Information Technology Department, the system serves to replace manual paper-based logbooks with a digitized database management module, improving administrative efficiency and reducing data redundancy.

At its core, the module handles essential member-related administrative tasks, including registration, profile updates, real-time roster searching, and record deletion. Through this interface, administrators can register members, link them to specific personal trainers (TrainorID) and membership options (PlanID), validate contact and demographic details, and manage active, pending, or expired membership statuses. The interface also supports asynchronous client-side search filtering to update active member listings dynamically.

To bridge the gap between a relational, normalized database and a responsive user interface, the system utilizes a modern web stack. User actions on the frontend trigger asynchronous JavaScript fetch operations, which communicate with backend PHP scripts that parse the inputs and execute prepared statements on a MySQL database. Validation occurs on both the client and server sides to protect structural data integrity, while a custom CSS layout ensures the user interface remains structured and readable.

2. Technology Stack

- **Frontend Framework:** Vanilla JavaScript (ES6+), utilising asynchronous Fetch API operations and dynamic DOM tree generation.
- **Styling:** Modern Custom CSS3 featuring flexible custom properties (:root variables), CSS Grid layout patterns, and transitions for form states.
- **Backend Framework:** Object-Oriented/Procedural PHP 8.x executing structural

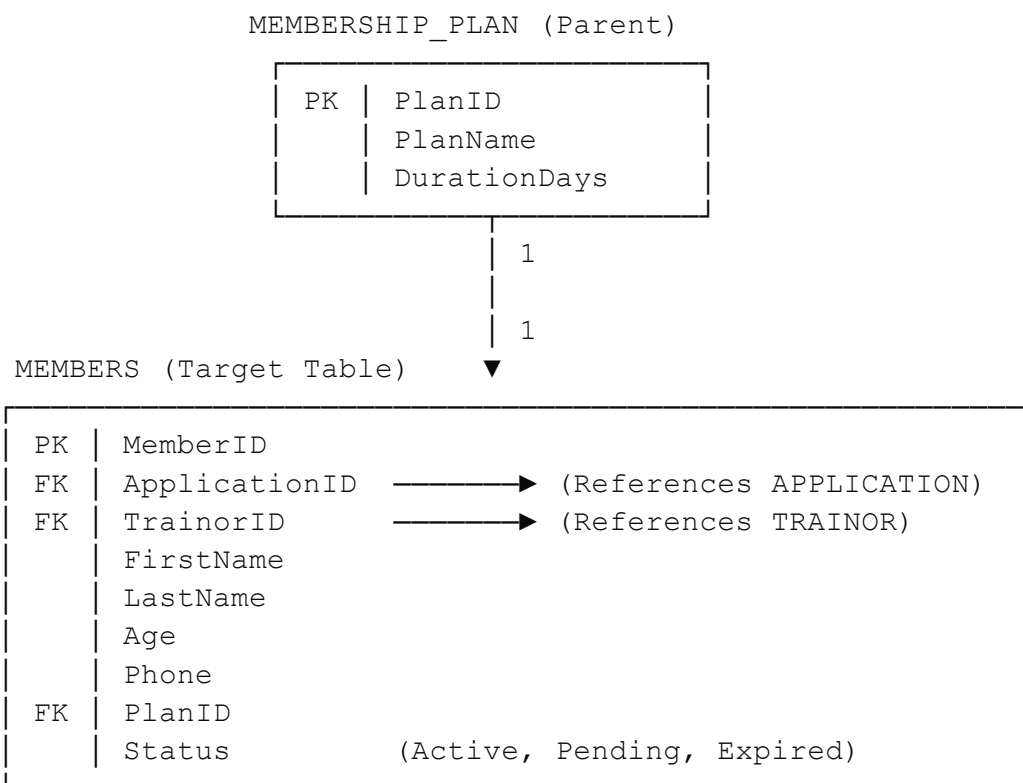
parameter bindings via MySQLi.

- **Database:** MySQL 8.x database with explicit InnoDB relationship constraints.
- **Additional APIs/Libraries:** Native JSON serialization engines and Browser Web APIs.

3. Database Design

Entity-Relationship Diagram (ERD) Subset

The database schema of FITMATE is fully normalized to 3NF. Below is the relational mapping of the specific tables managed or referenced by this CRUD module:



Relational Schema Properties (Data Dictionary)

- **MemberID** (INT(11), Primary Key, Auto-Increment, Required, Format: 999): Unique numerical identifier assigned automatically upon insertion.
- **ApplicationID** (INT(11), Foreign Key, Optional, Range: Nullable): Links the

registered member record back to their original enrollment application (application table).

- **TrainorID** (INT(11), Foreign Key, Optional, Range: Nullable): Maps the member to their assigned personal trainer (trainer table).
- **FirstName** (VARCHAR(100), Required, Format: Xxxxx): First name of the member.
- **LastName** (VARCHAR(100), Required, Format: Xxxxx): Last name of the member.
- **Age** (INT(2), Required, Range: 15 - 99): Age of the member.
- **Phone** (VARCHAR(12), Required, Format: 999999999999): Primary contact phone number.
- **PlanID** (INT(3), Foreign Key, Required, Format: 999): References the subscription plan type (membership_plan table).
- **Status** (VARCHAR(20), Required, Options: Active, Pending, Expired): The current state of the member's gym access.

4. Module Features

A. Create (Add Member)

Description: Administrators can add new members by filling out the profile card form. The frontend bundles the variables into a JSON object and POSTs it asynchronously to the backend.

Form Validation Rules:

FirstName and LastName must be non-empty strings.

Age must be a positive integer greater than or equal to 1.

Phone must be a valid numeric sequence.

PlanID is required to associate the member with a billing structure.

Status must be selected from the valid list of options (Active, Pending, Expired).

Error & Response Handling:

The backend (add_member.php) parses column fields dynamically, validating payload parameters prior to execution. If any required field is empty, a 400 Bad Request header is sent

along with a JSON error payload:

```
{"error": "The field 'FirstName' is required."}
```

If successful, the database returns a 201 Created status code, clears the edit state, clears all form inputs, and re-renders the table view.

B. Read (Active Roster list & Live Filter)

Description: The active gym rosters are displayed in an interactive list dynamically generated by JavaScript using templated string interpolation.

Live Filtering:

The search bar listens for a key-up event (onkeyup="searchMember()"). This pulls the current list of records, filters them on the client side using string matching against the member's first or last name, and updates the DOM immediately without requiring a full page refresh.

C. Update (Edit Member Info)

Description: Clicking the "Edit" button on a table row captures that record's data payload and populates the form editor fields. It assigns the record's primary key to a global edit state (window.currentEditId).

Updatable Validation & Rules:

Once edits are made, clicking "Update Member" triggers the updateMember() fetch sequence. If an administrator attempts to click update without selecting a member first, an alert is triggered: "Please click Edit first."

SQL Prepared Parameters:

The backend updates values using an exact prepared SQL template. To maintain referential integrity, blank inputs for optional attributes (such as ApplicationID and TrainorID) are explicitly converted to SQL NULL values:

```
$appID = !empty($data['ApplicationID']) ? $data['ApplicationID'] : null;
```

```
$trainID = !empty($data['TrainorID']) ? $data['TrainorID'] : null;
```

D. Delete (Remove Member Record)

Description: Removes a member record from the active roster using a parameter-bound delete operation.

Safety Measures:

To prevent accidental data loss, the delete action triggers a browser confirmation dialog:

```
if (!confirm("Delete this member?")) return;
```

Only upon explicit user confirmation does the program send a JSON-encoded request payload containing the targeted MemberID to the backend. The backend executes a prepared statement:

```
DELETE FROM members WHERE MemberID = ?
```

Upon success, the UI alerts the user and refreshes the table to reflect the current state.

6. Individual Reflections

Miko N. Ocfemia

Though this project was given a week prior to the deadline, we only had 2 days to accomplish it because we had little-to-no time to actually do it along with our other projects at the moment. I'd say the most difficult part of this entire project for me was trying to implement and learn the php codes. I was never familiar with it nor was it taught to us. So, after some help from others, we finally managed to implement a functional one to our chosen system.

With the little time we had. We only managed to implement the Create, Read, Update, and Delete (CRUD) components. We experienced a lot of errors trying to implement the other elements of the system.

The original system only had a few tables to it based on the original author's Entity Relationship Diagram. So, I decided to expand it from 9 tables to 12 tables. I added new tables that feel relevant to the topic, making it feel less lacking. But in our current functional system, there is only one. That being the members table with the functional CRUD components.

Trisha Mae M, Colina

The hardest part of understanding the original author's logic was figuring out how JavaScript and PHP and MySQL work together during the operation. At first, it was hard to follow the data flow from the Delete button in the user interface through the deleteMember () JavaScript function, to the file and finally to the database. I had to look at both the frontend and backend code to understand how the fetch () API sends JSON data and how PHP receives and processes that data using php://input. This was a challenge for me because I had to learn about JavaScript and PHP and MySQL and how they work together.

During the recreation process, I faced several challenges with the delete operation, including correctly passing the MemberID from the Delete button to the JavaScript function and handling JSON communication between JavaScript and PHP.

Given time I would make the Gym Member Management System better, I would fix errors often and make sure users get helpful messages and. I would replace those alert boxes with notifications that appear in the app. Also do add some animations that show when something is loading. Lastly would make sure the system logs errors on the backend better especially when deleting things. That way if something goes wrong I can see what happened.

Althea Stephanie P. Pura

Actually the difficult part is we have zero knowledge on php. Also the javascript is also hard since we're not yet that good since we just know the basic parts of it. We also find problems in connecting different separate files into a single working system. I think almost every part of our system needs further improvement as we've only focused on the member table. Even though we have created the html and css of the whole system we ended up creating a new one since we experienced hardship in connecting the files of the previous one.

Jon Wilfred C. Mayor

For this project, I was responsible for the Read and Search features. The most difficult part of the process was figuring out how to take the raw member data stored across different database tables and display it cleanly in one simple HTML table on

the dashboard. During coding, my biggest technical challenge was that typing in the search bar sent a database request on every single keystroke, which slowed down the app. I had to write JavaScript code that could filter the list of first and last names quickly without lagging or reloading the page. In the future, I would improve this module by adding a short delay so the search only triggers after the user stops typing, and by showing a "No members found" message if a search doesn't match any names.